STRICTLY CONFIDENTIAL • INTERNAL SECURITY AUDIT

Backend Threat & Vulnerability Report

Comprehensive Codebase Security Analysis for **The Sehat Buddy**

Platform: Node.js / Express / MongoDB
Assessment Date: October 2026
Auditor: Antigravity Security Analysis
Security Posture: HIGH EXPOSURE RISK

EXECUTIVE VERDICT: IMMEDIATE CRITICAL PATCHING REQUIRED

In its current state, the backend lacks foundational access barriers on critical management endpoints. Any unauthenticated actor on the public internet can register as platform administrator, wipe doctor or hospital profiles, hijack credentials through mass assignment, and flood Cloudinary media storage. Deployment to a public host would lead to rapid compromise.

3	4	5	3
CRITICAL EXPLOITS	HIGH-RISK FLAWS	MEDIUM EXPOSURES	HIPAA / COMPLIANCE GAPS

CRITICAL 1. Immediate Takeover & Destruction Vectors

1.1 Public Admin Registration Endpoint Without Authentication

`backend/routes/adminRoutes.js • POST /api/admin/register`

MECHANISM & THREAT

The route `router.post('/register', registerAdmin)` has no protection middleware, no shared secret, and no restriction. Anyone can send a raw POST request containing `{ "name": "Hacker", "email": "attacker@evil.com", "password": "pass" }` and obtain a signed Admin JWT token granting complete platform control.

> **Remediation:** Completely disable public admin registration, or enforce a private CLI seeding script / require master admin session authentication.

1.2 Unauthenticated Doctor & Hospital Modification & Deletion

`backend/routes/doctorRoutes.js & backend/routes/hospitalRoutes.js`

MECHANISM & THREAT

The following endpoints are completely unauthenticated: `PUT /api/doctors/:id`, `DELETE /api/doctors/:id`, `POST /api/doctors/add-by-hospital`, `PUT /api/hospitals/:id`, `DELETE /api/hospitals/:id`. Any anonymous user can delete entire doctor records or tamper with hospital listings simply by sending an HTTP request.

> **Remediation:** Mount `protect` and role-verification middleware (`verifyDoctorOrAdmin`, `verifyHospitalOrAdmin`) on all write/delete operations.

1.3 Account Hijacking via Mongoose Object Mass Assignment

`backend/controllers/doctorController.js & hospitalController.js`

MECHANISM & THREAT

Profile updates perform raw `Object.assign(doctor, req.body)` followed by `await doctor.save()`. Because Mongoose's `pre('save')` hook re-hashes any provided `password` field, an attacker can supply `{ "password": "newpass", "email": "takeover@attacker.com" }` to rewrite credentials and permanently seize control of any doctor or hospital account.

```
// VULNERABLE PATTERN IN PRODUCTION CODE:
Object.assign(doctor, req.body); // Directly injects untrusted fields
await doctor.save(); // pre('save') re-hashes and persists the attacker's password!
```

> **Remediation:** Implement strict field whitelisting. Only extract permitted keys (e.g. `const { bio, experience, contactNumber } = req.body;`). Never pass raw `req.body` to models.

HIGH 2. Authentication, Authorization & Data Exposure Risks

2.1 Broken Authorization & Absence of Role-Based Access Control (RBAC)

`backend/middleware/authMiddleware.js`

MECHANISM & THREAT

The `protect` middleware solely decodes the JWT and attaches `req.user = { id: decoded.id }`. It does not check user roles (`admin`, `doctor`, `hospital`, `patient`) and does not verify if the user still exists or is active in MongoDB. Any authenticated user (e.g., standard patient) can call `/api/admin/dashboard` or doctor management endpoints.

> **Remediation:** Introduce role validation checks (e.g. `authorize('admin')`) and verify the account state in MongoDB on protected operations.

2.2 Plaintext OTP Logging & Infinite Brute-Force Feasibility

`backend/controllers/doctorController.js & hospitalController.js`

MECHANISM & THREAT

1. Plaintext OTPs are outputted to stdout: `console.log(`[DEV OTP] ...: ${otp}`)`, creating risk in centralized log pipelines.
2. 6-digit OTPs have only 900,000 combinations. The reset endpoints have **no attempt counter**, **no lockout**, and **no rate limiting**. An attacker can brute-force the 6-digit OTP well within its 10-minute validity window.

> **Remediation:** Hash OTPs before saving to DB, enforce a maximum limit of 3 failed attempts before voiding the token, and remove console logging.

2.3 Password Hash Exposure in Model Payloads

`backend/models/Doctor.js, Hospital.js, User.js`

MECHANISM & THREAT

The `password` schema fields omit `select: false`. Whenever full profile documents are returned to the client (such as during login, profile queries, or list fetching), the raw bcrypt password hash is transmitted over the wire, leaving hashes vulnerable to interception and offline cracking.

> **Remediation:** Add `select: false` to the `password` definition in all Mongoose schemas and strip sensitive fields prior to JSON serialization.

2.4 Unrestricted Media Uploads & Denial-of-Service (DoS)

`backend/routes/uploadRoutes.js • 250MB File Limit`

MECHANISM & THREAT

Endpoints `/api/upload/image`, `/api/upload/video`, and `/api/upload/base64` are completely public. The video upload route accepts up to 250MB per file and buffers it to disk. Attackers can flood the endpoint, filling the server's hard disk and exhausting monthly Cloudinary bandwidth and storage limits.

> **Remediation:** Enforce authentication on all upload endpoints, reduce maximum payload sizes, and enforce per-user rate limits.

MEDIUM 3. Infrastructure & Operational Weaknesses

Area	Identified Vulnerability	Mitigation
HTTP Headers	No `helmet` middleware. Missing `X-Frame-Options`, `Content-Security-Policy`, and HSTS, enabling clickjacking and MIME attacks.	Install `helmet` and invoke `app.use(helmet())` in `server.js`.
CORS Configuration	Permissive check with fallback to wildcard origins combined with `credentials: true`.	Strictly bind CORS to verified frontend production origins only.
Brute-Force Protection	Zero rate limiting across `/login` endpoints for users, doctors, hospitals, and admins.	Apply `express-rate-limit` (e.g. max 5 login attempts per 15 minutes per IP).
Error Handling	No global `(err, req, res, next)` middleware. Raw `error.message` leaks DB stack traces and schema validation keys.	Implement centralized error boundary that logs details internally and returns clean sanitized errors to clients.
Input Validation	Ad-hoc validation without schema enforcement (Zod/Joi). Unsanitized parameters flow into database operations.	Adopt standardized request body schema validation.

COMPLIANCE 4. Healthcare Privacy & Compliance (HIPAA / Telemedicine)

Because **The Sehat Buddy** handles sensitive medical consultations, doctor availability schedules, hospital partnerships, and patient queries, standard commercial compliance frameworks (e.g. HIPAA Security Rule, DISHA, GDPR) mandate higher security controls:

4.1 Absence of Immutable Audit Trails

There is currently no audit logging mechanism recording *who* modified, viewed, or deleted a doctor or hospital profile. Healthcare standards require tamper-evident logging of all PHI (Protected Health Information) and staff credential modifications.

4.2 Lack of Active Token Revocation / Invalidation

JWT tokens are purely stateless with no server-side blacklist or token version tracking. If a doctor or admin is terminated or credentials are breached, active tokens remain valid until expiration with no immediate kill-switch capability.

4.3 Medical Data at Rest

Ensure production MongoDB instances utilize encryption-at-rest (AES-256) and TLS 1.3 for all database connection strings.

ACTION PLAN 5. Priority Remediation Roadmap

- **Phase 1: Emergency Access Lockdown (Immediate / Day 1)**
  - Remove public `/api/admin/register` endpoint.
  - Attach `protect` middleware to `PUT /api/doctors/:id`, `DELETE /api/doctors/:id`, and all `/api/hospitals` routes.
  - Replace `Object.assign(doc, req.body)` with strict attribute whitelisting in doctor and hospital controllers.
- **Phase 2: Authentication & RBAC Hardening (Day 2 - 3)**
  - Implement role-based middleware: `requireRole(['admin'])`, `requireRole(['doctor', 'admin'])`.
  - Add `select: false` to passwords in all Mongoose models.
  - Enforce OTP max attempts (3 strikes) and remove plaintext console logging.
- **Phase 3: Rate Limiting & Upload Quotas (Day 4)**
  - Integrate `express-rate-limit` for all login, password reset, and registration endpoints.
  - Restrict media upload routes to authenticated sessions and reduce video file upload caps.
- **Phase 4: Defense-in-Depth & Production Hardening (Day 5)**
  - Add `helmet` security headers and tighten CORS whitelists.
  - Establish global centralized error handler to prevent internal stack trace leakage.